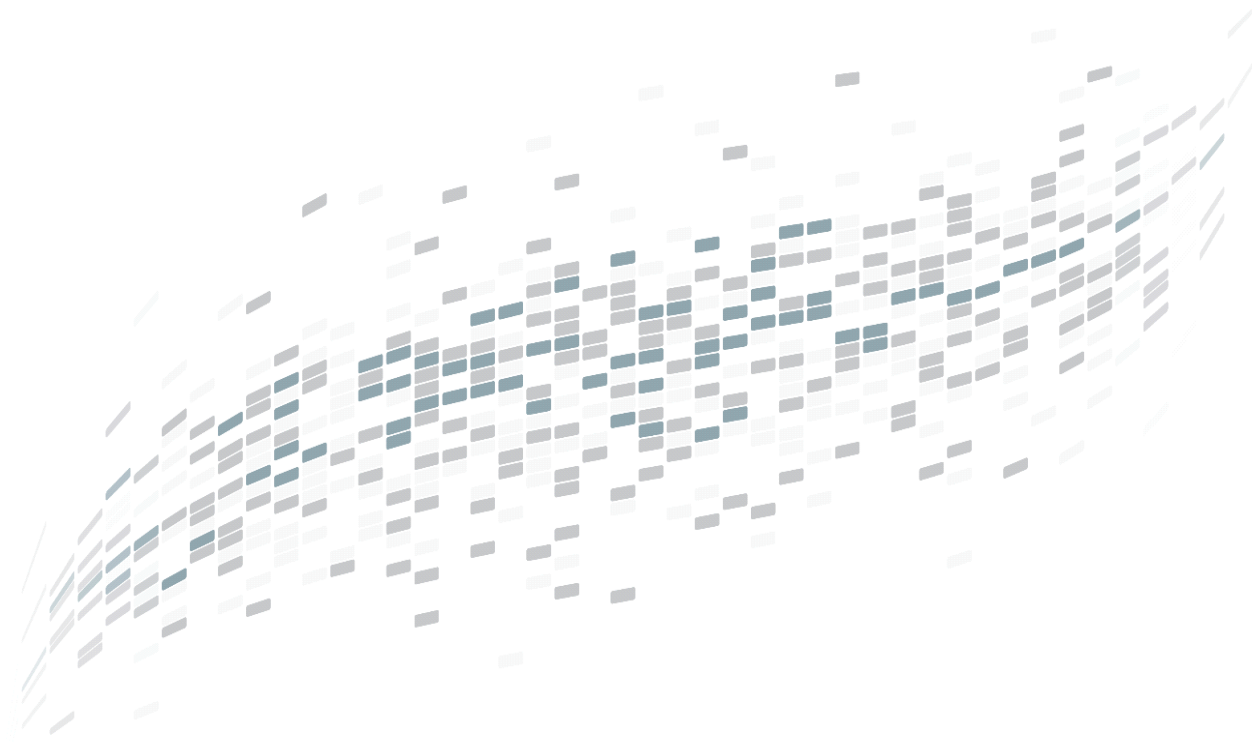




DOIP UDS Server User Guide



Copyright

This document is property of Visteon Corporation.

The information in this document is confidential!

Copyright © 2023, Visteon. All rights reserved.

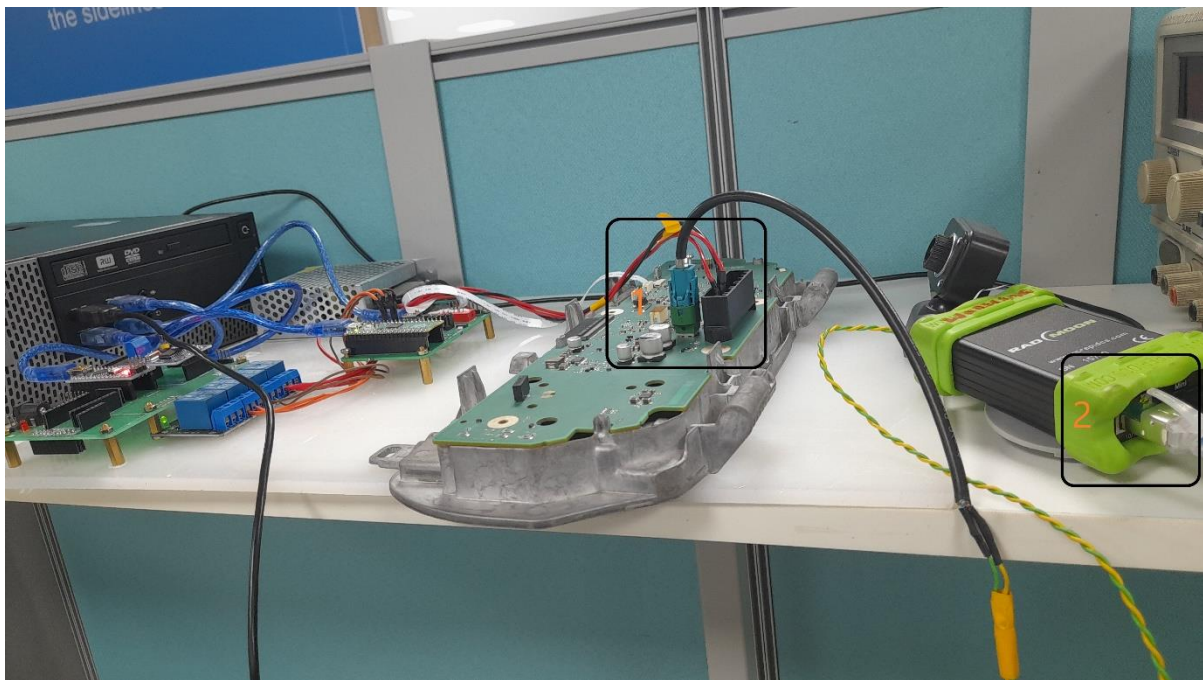
Introduction

- This document describes the installation, setup, and execution of DoIP UDS server stack.

Hardware setup

In below pic shows the proper working setup

- 1 show that Ethernet should be connected to the setup
- 2 shows that LAN connection from radmoon should be connected to ubuntu pc /Windows PC.



(Note : If connected to windows pc , virtual box should be ran in that pc and LAN should be connected)

Execution steps

Once connected check the ifconfig to list the interface configurations. In our case LAN is listed as **enx000e09882467**

```
ubuntu@IND9PDI72-L:~/Desktop/Gautham$ ifconfig
docker0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    inet6 fe80::42:5ff:fe0a:c9af prefixlen 64 scopeid 0x20<link>
    ether 02:42:05:0a:c9:af txqueuelen 0 (Ethernet)
    RX packets 382 bytes 30412 (30.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1780 bytes 387782 (387.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

enp0s25: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.138.207.114 netmask 255.255.255.0 broadcast 10.138.207.255
    inet6 fe80::3593:cf7a:5149:a2a0 prefixlen 64 scopeid 0x20<link>
    ether 64:00:6a:6e:23:ee txqueuelen 1000 (Ethernet)
    RX packets 27526881 bytes 37977480350 (37.9 GB)
    RX errors 0 dropped 1645 overruns 0 frame 0
    TX packets 3132550 bytes 1224454667 (1.2 GB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 20 memory 0xfbl00000-fbl20000

enx000e09882467: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    ether 00:0e:09:88:24:67 txqueuelen 1000 (Ethernet)
    RX packets 12 bytes 894 (894.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3612 bytes 637778 (637.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 16387 bytes 1876118 (1.8 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 16387 bytes 1876118 (1.8 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

veth7fe9b5d: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::acf9:27ff:fe69:a910 prefixlen 64 scopeid 0x20<link>
    ether ae:f9:27:69:a9:10 txqueuelen 0 (Ethernet)
    RX packets 382 bytes 35760 (35.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2010 bytes 406055 (406.0 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Creating Vlan

- Once listed down we will create vlan with **enx000e09882467** as parent interface.
- Commands to create vlan listed below

```
"sudo ip link add link enx000e09882467 name vlan110 type vlan id 110
sudo ip link set vlan110 address aa:bb:cc:dd:00:0a
sudo ip addr add 192.168.11.10 dev vlan110
sudo ip link set vlan110 up
sudo ifconfig vlan110 broadcast 192.168.11.255
sudo ifconfig vlan110 netmask 0xfffff00"
```

- Vlan110 is created as shown below

```

docker0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    inet6 fe80::42:5ff:fe0a:c9af prefixlen 64 scopeid 0x20<link>
    ether 02:42:05:0a:c9:af txqueuelen 0 (Ethernet)
    RX packets 382 bytes 30412 (30.4 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1783 bytes 388422 (388.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

enp0s25: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.138.207.114 netmask 255.255.255.0 broadcast 10.138.207.255
    inet6 fe80::3593:cf7a:5149:a2a0 prefixlen 64 scopeid 0x20<link>
    ether 64:00:6a:6e:23:ee txqueuelen 1000 (Ethernet)
    RX packets 27527831 bytes 37977569926 (37.9 GB)
    RX errors 0 dropped 1645 overruns 0 frame 0
    TX packets 3132652 bytes 1224469514 (1.2 GB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
    device interrupt 20 memory 0xfb100000-fb120000

enx000e09882467: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::d754:540b:9ce1:dc0e prefixlen 64 scopeid 0x20<link>
    ether 00:0e:09:88:24:67 txqueuelen 1000 (Ethernet)
    RX packets 12 bytes 894 (894.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3707 bytes 654400 (654.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 16463 bytes 1882462 (1.8 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 16463 bytes 1882462 (1.8 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

veth7fe9b5d: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet6 fe80::acf9:27ff:fe69:a910 prefixlen 64 scopeid 0x20<link>
    ether ae:f9:27:69:a9:10 txqueuelen 0 (Ethernet)
    RX packets 382 bytes 35760 (35.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2013 bytes 406695 (406.6 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

vlan110: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.11.10 netmask 255.255.255.0 broadcast 192.168.11.255
    inet6 fe80::a8bb:c0ff:fedd:a prefixlen 64 scopeid 0x20<link>
    ether aa:bb:cc:dd:00:0a txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 73 bytes 12509 (12.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

ubuntu@IND9PD1T72-L:~/Desktop/Gautam$

```

- Make sure to disable the firewall (pfctl -d)
- Enable the arp for the interface which you're going to test (ifconfig vlan11 arp)
- Ping the target and vice versa. U should be able to ping it

```
ubuntu@IND9PD1T72-L:~/Desktop/Gautham$ ping 192.168.11.2
PING 192.168.11.2 (192.168.11.2) 56(84) bytes of data.
64 bytes from 192.168.11.2: icmp_seq=1 ttl=255 time=1.66 ms
64 bytes from 192.168.11.2: icmp_seq=2 ttl=255 time=0.926 ms
64 bytes from 192.168.11.2: icmp_seq=3 ttl=255 time=0.893 ms
64 bytes from 192.168.11.2: icmp_seq=4 ttl=255 time=0.862 ms
64 bytes from 192.168.11.2: icmp_seq=5 ttl=255 time=0.925 ms
```

Compilation of test code

Syntax

- `g++ -pthread DoIP_DiagTester.cpp -o <output executable>`

Example

- `g++ -pthread DoIP_DiagTester.cpp -o Doip_test`

Then run the executable target file with `sudo`

- `sudo ./Doip_test`

Update the details when requested,

- You can check below screenshot , to understand the testing

```
Enter target physical logical address:
#it is the DoIPTargetAddressPhysical
#configured in uds.ini at the server
#Ex: 006F
006F
6f

Enter source address:
#it is one of the known DoIPSourceAddress_0...N
#configured in uds.ini at the server
#Ex: E000
0F00
f00
Doip Tester Instance creation : Success

Enter source interface name:
#It is the PC tester ethernet interface name
#to which server is connected.
#Ex: enol
vlan110

iface name is vlan110
UDP socket creation successful3
UDP Socket : Bind to device failed

Enter source interface name:
#It is the PC tester ethernet interface name
#to which server is connected.
#Ex: enol
^C
ubuntu@IND9PD1T72-L:~/Desktop/Gautham$ sudo ./Doip

Enter target physical logical address:
#it is the DoIPTargetAddressPhysical
#configured in uds.ini at the server
#Ex: 006F
006F
6f

Enter source address:
#it is one of the known DoIPSourceAddress_0...N
#configured in uds.ini at the server
#Ex: E000
0F00
f00
Doip Tester Instance creation : Success

Enter source interface name:
#It is the PC tester ethernet interface name
#to which server is connected.
#Ex: enol
vlan110

iface name is vlan110
UDP socket creation successful3
UDP Bind to device success 3
UDP broadcast permission set...3
UDP_Socket_Init Success : migrating to next state >> UDP_Socket_Completed

Enter below to choose vehicle identification request message type:
1 for vehicle identification request message
2 for vehicle identification request message with VIN
3 for vehicle identification request message with EID
```

```

Enter below to choose vehicle identification request message type:
1 for vehicle identification request message
2 for vehicle identification request message with VIN
3 for vehicle identification request message with EID
1
idReq1

SIZE OF UDP SENT DATA FROM CLIENT =8
2      FD      0      1      0      0      0      0

UDP Vehicle Send Success

Message received at Doip Tester <UDP >:Length(inBytes)=41
DoipTester_ProcessUdpData : Vehicle Response Received
    02 FD 00 04 00 00 00 21 01 02 03 04 05 06 07 08      .....!.....
    09 0A 0B 0C 0D 0E 0F 10 11 00 6F AA BB CC DD 00      .....0.....
    02 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....

#####
TCP Socket creation Success..

Enter Server TCP IP address:
#Ex: 192.168.1.76
192.168.11.2

Entered IP address is: 192.168.11.2
TCP connection success..
TCP Success : migrating to next state >> TCP_Socket_Completed

Enter routing activation type:
#It is DoIPRoutingactivationType
#configured in doip.ini at the server
#00 Default
#01 WWH-OBD
#E0 Central security
0
TCP Routing Activation Req Send...

SIZE OF TCP ROUTING ACTIVATION SENT DATA FROM CLIENT =19
2 FD 0 5 0 0 0 B F 0 0 0 0 0 0 0 0 0 0 0

Message received at Doip Tester <TCP>:Length(inBytes)=21
Logical address of the ECU f00
Logical address of the external test equipment 6f
Routing activation Response received
Routing activation Status value 10
Routing_Activation_Completed
    02 FD 00 06 00 00 00 0D 0F 00 00 6F 10 00 00 00      .....0....
    00 00 00 00 00 00 00 00 00 00 00 00 00 00 00      .....

```

```

Routing_Activation_Completed
 02 FD 00 06 00 00 0D 0F 00 00 6F 10 00 00 00 .....O....
00 00 00 00 00 .....

#####
Press 'A' for Diagnostic Session Service $0x10: Default
Press 'C' for Diagnostic Session Service $0x10: Extendedd
Press 'D' for Security Access Service $0x27
Press 'G' for Tester Presence Service $0x3E
Press 'H' to send target data (Max Length check)
Press 'I' for ECU Reset Service $0x11
Press 'L' for Read Data By Identifier Service $0x27
Press 'M' to send minimum data (Min Length Check)
Press 'B' for request download $0x34
Press 'F' for data transfer $0x36
Press 'T' for exit data transfer $0x37
Press 'Z' to exit gracefully
A
doipstate
Diagnostic Session e

SIZE OF TCP DATA FROM CLIENT = 14
2 printed

Message received at Doip Tester <TCP>:Length(inBytes)=13
Positive Diag Ack Received : 0x8002
Logical address of the ECU 6f
Logical address of the external test equipment f00
Diagnostic message Pos Ack received 0
 02 FD 80 02 00 00 05 00 6F 0F 00 00 .....O...

#####

Message received at Doip Tester <TCP>:Length(inBytes)=18
Logical address of the ECU 6f
Logical address of the external test equipment f00
Diag Session Response Received : 0x50
Diag Session is50
 02 FD 80 01 00 00 00 0A 00 6F 0F 00 50 01 00 32 .....O..P..2
01 F4 ..

```

In the path ,

- programs\nissan\my2022\j32v\gp-build\prebuilt_common\etc\doip.ini
- programs\nissan\my2022\j32v\gp-build\prebuilt_common\etc\uds_core.ini

This file can be modified and tested as same. With Wireshark it is possible to store the logs as well.